Ex2 Thursday (Day 1): full guide

One small step at a time. Do the step, tick it, move on. Do not read ahead. Today: **A. Malware** and **B. Threats and attacks**. Tomorrow: Autopsy, assurance, legal, justification.

Rules from the older v4 example pack. Check each against your live brief.

Open book: notes, books and the internet are allowed, **but "no personal communication and or on-line asynchronous interaction permitted"**. It is an individual test. Do not message anyone, and do not use an AI chat, during test hours. This guide is written to be used on its own.

Tests run **09:30 to 16:00**. Active time is 6 hours. Take **one 30-minute break** and **email your supervisor when it starts and when it ends**.

If screenshots are requested: **maximum 5 pages of A4**. Crop tightly.

At the end of each test day, all work must be saved and uploaded as instructed. **Submission is final and cannot be reloaded or amended.**

If you feel stuck:

1. Stand up, drink water, look away from the screen.
2. Say aloud: "The next step is ____."
3. Write your assumption in one line, take a screenshot, move on. Come back at the end.

What Merit needs (v4 Annex 1; the live brief points to supplementary tables, so check those): every pass criterion, plus Merit on items **9 and 12**. So the steps marked **MERIT** below are not optional if you want Merit.

Item	Pass	Merit	Distinction
9 Malware	Analyse malware, identify its mechanisms	Malware with **obfuscation and anti-reverse-engineering**	**Script or compose custom tool sets** for complex malware
12 Threats	Research and test attacks in a lab, combine sources into an enriched view	Threats to the **system, services and processes**	**Devise mitigations** for all of those

1. Start of day

Folders and notes

☐ Make a folder **`Ex2_Day1`**. Inside: **`screenshots`**, **`outputs`**, **`notes`**.

☐ Open **`notes/evidence_log.txt`**. After every step add one line: figure number, what it shows, file name.

☐ Name screenshots in the order you will use them: **`fig01_sample_hash.png`**, **`fig02_triage_report.png`**.

☐ Open the Snipping Tool (**`Win+Shift+S`**).

☐ **Screenshot budget:** max 5 pages of A4 if screenshots are requested. Crop tightly. Keep a running **`evidence.docx`** of cropped images with captions so you can check the page count as you go. Log the rest in your notes only.

Read the brief

☐ Read the whole Ex2 brief once. Touch nothing.

☐ Highlight every verb (analyse, identify, test, justify, develop).

☐ Note which topics are highlighted yellow. Those are mandatory.

Rules of engagement

Write this on the first page of **`notes/`** (it is part of the legal and ethics topic):

☐ Authorised by: me, for my own lab, for my own coursework.

☐ In scope: my Ex1 prototype app, my lab VMs, one malware sample inside an isolated VM, the M57-Jean image.

☐ Out of scope: any employer or CareLink system, any live site, anyone else's machine.

☐ Safety: VM snapshots before each test, revert after. No real personal data.

☐ Stop and escalate if anything touches something out of scope.

Get things running in the background. This saves time later.

☐ Start the lab VMs. Take a snapshot of each. Name it **`clean`**.

☐ Start your Ex1 prototype (Apache and MySQL). Follow **`ex1-design/prototype/README.md`**. Run **`bash test_sync.sh`** once. Expect 10 PASS.

☐ Note the IP of the machine running the app. From another VM use that IP, not **`localhost`**. Call it **`<TARGET>`**.

☐ Open Nessus (**`https://localhost:8834`**). Start the activation or plugin setup now. Come back to it in B3.

☐ Get the **sample** (if not done already): log in to VirusShare in the lab VM only, download your chosen file. Never on a work laptop.

☐ Get the **M57-Jean image** for tomorrow. Record its SHA-256 (**`Get-FileHash -Algorithm SHA256 <file>`**). Do not open it today.

2. A. Malware

Done when: you have a verdict, ATT&CK techniques, IOCs, a detection rule, what CareLink should do, and (for Merit) obfuscation and anti-reverse-engineering analysis.

Time inside A: A1 10, A2 30, A3 40, A4 10, A5 20, A6 10.

A1. Safe setup

☐ Lab VM only. Not your laptop, not the host.

☐ Snapshot **`clean`** exists. Network on **host-only or off** for static work.

☐ Defender may quarantine the download. Use an agreed exclusion folder inside the VM.

☐ Unzip. Zip password is usually **`infected`**: **`7z x sample.zip -pinfected`** (or **`unzip -P infected sample.zip`**).

☐ Rename to **`sample.bin`**. Never double-click it.

☐ Hash it:

Windows: **`Get-FileHash -Algorithm SHA256 .\sample.bin`**

Linux: **`sha256sum sample.bin`**

☐ Write the SHA-256 in your evidence log. Screenshot it (**`fig01`**).

☐ Search that hash on VirusTotal. **Do not upload the file.** Note the family name and the detection count. Screenshot (**`fig02`**).

☐ In one line, say why this sample suits CareLink. Example: “Infostealers target saved credentials, like the sign-in details cached on 840 devices.”

A2. Static analysis

The sample is **not run** in this step.

☐ Run the triage tool: **`python3 ex2-malware/triage.py sample.bin --yara ex2-malware/rules --md report.md --json report.json`**

☐ Open **`report.md`**. Screenshot the top (hashes, type, size, entropy) as **`fig03`** and the imports and ATT&CK table as **`fig04`**.

☐ Note these in your evidence log:

☐ File type and size

☐ MD5, SHA-1, SHA-256

☐ Compile timestamp (can be faked, say so)

☐ Sections and entropy. Above about 7 suggests packed or encrypted.

☐ Interesting imports (injection, persistence, network, anti-debug)

☐ YARA hits

☐ Strings: **`strings -n 8 sample.bin > notes/strings.txt`** (Windows: **`strings64.exe -n 8 sample.bin`**). Search it for: **`http`**, **`.exe`**, **`.dll`**, **`HKEY`**, **`Run`**, **`cmd`**, **`powershell`**, IP addresses.

☐ Copy any URLs, IPs, domains, registry keys and file paths onto the IOC table.

☐ If few imports and high entropy, write: “likely packed, so static view is limited”.

☐ Remember: **imports show capability, not proof of use.** Write that sentence in your notes.

☐ Optional if installed: Detect It Easy for packer and entropy, **`floss sample.bin`** for hidden strings.

☐ **DECISION POINT (Merit).** Does the sample show packing, high entropy, few imports, dynamic API resolution (**`GetProcAddress`**), or evasion strings? Yes: keep it, this is Merit material. No: swap to a more obfuscated sample now. Then carry on.

A3. Dynamic analysis

Plan before you click. Free sandbox time is short.

☐ Write your 4 questions: What processes does it start? What files and registry does it change? Where does it connect? Does it persist?

☐ Any.run is the tool the v4 brief names. Observe the behaviour in the **interactive report** and use it to decide the type and purpose. New task, upload **`sample.bin`**, pick a Windows option that suits the file (32 or 64-bit), set a run time (check the free-tier limits on the page).

☐ Know that free analyses are, I believe, public. Fine for a public sample. Never upload your own files or evidence.

☐ If Any.run will not run it, use your own CAPEv2 sandbox. Say in the write-up which you used and why.

☐ While it runs, screenshot:

☐ Process tree (**`fig05`**)

☐ Files created or changed, and registry keys, especially Run keys (**`fig06`**)

☐ Network: DNS names, IPs, HTTP requests (**`fig07`**)

☐ The ATT&CK or behaviour summary if shown (**`fig08`**)

☐ Copy IOCs from the report to your IOC table. Labels in the tool may differ: look for network, files, registry and IOC views.

☐ Note anything that looked like evasion: long sleeps, checks for a VM or debugger, nothing happening at all.

☐ Export or save the report if the tool offers it. Record the report link or file name.

A4. Compare static and dynamic

☐ Fill the **Static vs dynamic** table in your notes: agreed, only static, only dynamic.

☐ Write two lines: where they agree, and where they differ and why.

☐ Mark each ATT&CK technique as **confirmed** (seen in sandbox) or **suspected** (only imports or strings).

A5. Verdict, detection and CareLink actions

☐ Verdict in one line: type (stealer, loader, RAT, ransomware), purpose, confidence.

☐ How it gets in (phishing email, malicious download), how it runs, how it persists, how it talks out, how it hides. Use those five headings.

☐ Write a YARA rule:

1. Copy a rule from **`ex2-malware/rules/generic_triage.yar`**.
2. Change the strings to ones from **your** sample.
3. Test it hits: **`yara -s myrule.yar sample.bin`**
4. Test it misses a clean file: **`yara myrule.yar C:\Windows\System32\notepad.exe`** (Linux: **`/bin/ls`**)
5. Screenshot both (**`fig09`**). Write what you changed.

☐ Detection, containment, mitigation for CareLink. For each, link to something already in your Ex1 design:

☐ Block the IOCs (DNS, firewall)

☐ EDR or Defender rule from your YARA

☐ Short-lived tokens and MFA limit what a stolen credential can do

☐ Small cache limits what a stealer can find

☐ Isolate the device, revoke its token, remote wipe

☐ Warn staff about the delivery route

☐ Revert the VM to **`clean`**.

A6. MERIT: obfuscation and anti-reverse-engineering

☐ List what you saw, for each of these, or write “not seen”:

☐ Packing or encryption (entropy, few imports)

☐ Dynamic API resolution (**`LoadLibrary`** and **`GetProcAddress`**)

☐ Anti-debugging (**`IsDebuggerPresent`**, timing checks)

☐ Anti-VM or anti-sandbox (VM strings, long sleeps, nothing happening)

☐ For each one seen: **how you got past it** (read behaviour in the sandbox after unpacking, dumped memory, waited out a sleep, found the decoded strings).

☐ Be ready to say in two lines each: the machine-level view (what the instructions do), what reverse engineering means here, what de-obfuscation means, and how anti-debugging works.

☐ **DISTINCTION (only if time):** script or compose your own tool set. Make a small, visible change to **`triage.py`** (for example, a check for VM-detection strings) or chain tools into one run. Write down exactly what you changed. Say plainly that the base tool is pre-written.

Break

Stand up. Water. Eat something. Look away from the screen for a bit.

3. B1. Intel

Done when: you have a short method, a sources list and 3 realistic threats for CareLink.

☐ Write the method in four words: **collect, check reliability, analyse, share.**

☐ List sources you would use and rate each A (reliable), B (usually), C (check it):

☐ NCSC (advice, alerts)

☐ CISA Known Exploited Vulnerabilities catalogue

☐ MITRE ATT&CK

☐ OWASP Top 10 (for the web app)

☐ Vendor reports for your malware family

☐ NHS England cyber alerts

☐ One line on dark web and social media: passive, lawful monitoring only, never engage.

☐ Pick **3 threats**, one technical, one human, one physical. Suggested:

1. Infostealer or ransomware on a staff device (from A).
2. Phishing that steals a sign-in and then a token.
3. Lost or stolen device holding cached visit notes.

☐ For each: asset, how it happens, impact, which Ex1 control helps.

☐ **Demonstrate collecting intelligence:** show 2 or 3 real searches on public sources (security news, researcher posts, vendor reports). Passive and lawful only: never join, post, buy or log in to a forum. For dark web activity, use published threat-intel reporting, do not visit.

☐ **Enriched view (Pass):** combine at least two sources in one small table: threat | what NCSC says | what CISA KEV or ATT&CK says | what it means for CareLink.

☐ Screenshot or save one real source for each (a CISA entry, an ATT&CK page).

4. B2. Threat model

Done when: one diagram and one STRIDE table.

☐ Draw the data flow of the Ex1 sync app (paper is fine):

External entity: clinician **device and app**

Flow 1: HTTPS from device to API

Process: **sync.php** API

Flow 2: SQL from API to database

Data stores: **notes** table, **api_tokens** table

Second entity: **IT admin**, who issues and revokes tokens

☐ Draw dashed lines for **trust boundaries**: device | internet | server | database.

☐ Label each flow with the protocol and the data (token, note).

☐ Fill the STRIDE table. The brief says the model must be based on your intelligence, so add a fifth column **Intel**: which B1 source or finding each threat came from. Start from these and edit:

S/T/R/I/D/E	Threat	Existing control	How you test it
S	Stolen or guessed token posts fake notes	Hashed, scoped, expiring, revocable tokens	B4 brute force, revoke test

S/T/R/I/D/E	Threat	Existing control	How you test it
T	Note changed in transit	TLS only	Wireshark HTTP vs HTTPS
R	Staff deny writing a note	**`staff_id`**, **`created_at`**, logs without note text	Check a stored row
I	Error leaks SQL	**`display_errors`** off, generic 500	Tests 7 and 10, SQLMap
I	Cached notes read on a lost device	Disk encryption, PIN, small cache	OS hardening evidence
D	Flood of requests	64 KB cap, 413	B4 flood test (finding: no rate limit)
E	Read-only token writes	Scope check returns 403	Test 5
E	SQL injection reaches database	Prepared statements	SQLMap

☐ Mark the 3 highest risks (likelihood x impact, 1 to 5 each, same scale as Ex1).

☐ Screenshot or photograph the diagram (**`fig10`**) and the table (**`fig11`**).

5. B3. Scan your own app

Only your own app, only in your lab. Target is **`<TARGET>`**, the machine running Apache.

☐ Snapshot or export the database first.

☐ **Nessus (start first, let it run):** New Scan, Basic Network Scan, target **`<TARGET>`**, save, launch. Do the others while it runs.

☐ **nmap:** **`nmap -sV -sC -p- -oN outputs/nmap.txt <TARGET>`**. Screenshot open ports and versions (**`fig12`**). Note anything that should not be open (MySQL 3306, FTP, SMB).

☐ **Nikto:** **`nikto -h http://<TARGET>/prototype/ -o outputs/nikto.txt`**. Note headers and files it flags (**`fig13`**).

☐ **dirb:** **`dirb http://<TARGET>/prototype/ -o outputs/dirb.txt`**. Note any file that should not be public, like **`db.php`** or backups (**`fig14`**).

☐ **Burp:**

1. Proxy tab, Intercept on, “Open browser”.
2. Send one valid request with **`curl -x http://127.0.0.1:8080 ...`** or the test script through the proxy.
3. Right-click the request, send to **Repeater**, change the token, send again.
4. Screenshot the request and response (**`fig15`**).

☐ When Nessus finishes: export the report. Record the top findings and severities (**`fig16`**).

☐ For each tool, write one line: what it found, whether it is real or a false positive, and what you would do.

6. B4. Attack your own app

Snapshot the database before you start. Tests add rows. Clean up after.

Set your variables (use your real values):

```
TARGET=<TARGET>
URL=http://$TARGET/prototype/sync.php
TOKEN=<write token>
```

Attack 1: SQL injection with SQLMap

☐ Run:

```
sqlmap -u "$URL" --method POST \
 --data '{"client_id":"sq1","staff_id":"s1","patient_ref":"P1*","note_text":"x","created_at":"20
26-11-12T09:00:00Z"}' \
 -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
 --batch --level 3 --risk 1
```

☐ Expect: “does not appear to be injectable” because of prepared statements. That is a good result. Screenshot it (**`fig17`**).

☐ Write: worked? **No.** Impact if it had: read or change all notes. Why it held: parameterised queries. Residual: other endpoints, if any.

Attack 2: guess the token

☐ Run 200 wrong tokens and count the responses:

```
for i in $(seq 1 200); do curl -s -o /dev/null -w "%{http_code}\n" -X POST \
 -H "Authorization: Bearer guess-$(printf %016d $i)" -H "Content-Type: application/json" \
 -d '{}' "$URL"; done | sort | uniq -c
```

☐ Expect: 200 x **`401`**. Screenshot (**`fig18`**).

☐ Finding: the endpoint answers every guess with no lockout or rate limit. Guessing is not realistic because tokens are long and random (check **`make_token.php`**), but the endpoint can be hammered.

☐ Fix: rate limit per IP and per token, alert on repeated 401s, tuned so it does not cause alert fatigue.

Attack 3: abuse cases

Fill in the table in your notes. Use **`curl`** or Burp Repeater.

Test	Expect	What it proves
Read-only token tries to write	403	Authorisation works
Revoke a token in the database, then use it	401	Revocation works
Expired token	401	Expiry works
Same note sent twice	200, one row only	Safe to repeat
3 MB note	400 or 413	Size limit works
Malformed JSON	400, no SQL in the response	No information leak
`GET` instead of **`POST`**	405	Method check
Token sent over plain HTTP	Works if **`REQUIRE_HTTPS`** is false	Finding: set it true and retest

☐ “IDOR or bad upload” in the brief: this API has no read or upload function. Write “not applicable” and the reason. Do not invent a result.

☐ Pick **one** fix, apply it, and rerun the test to show it worked. For example, set **`REQUIRE_HTTPS`** to true and show HTTP now gets 403 and HTTPS still works.

☐ Clean up: delete the test rows or restore the database snapshot.

☐ Revert VMs to **`clean`**.

B5. MERIT: threats to services and processes

Merit means threats to the system **including services and processes**, not just technology.

☐ List threats to **processes**:

☐ Helpdesk password or MFA reset by someone pretending to be staff

☐ Supplier or cloud vendor with admin access

☐ Leavers keeping tokens or devices

☐ A staff member enabling an attack without knowing it (for example, opening a phish)

☐ **DISTINCTION:** a mitigation for each:

☐ Call-back to a known number plus manager approval before any reset

☐ Supplier access just-in-time, logged, with a named owner and contract clause

☐ Leaver checklist that revokes tokens and wipes the device on day one

☐ Phishing reporting route and short staff training

7. Tidy, evidence log, upload

☐ Evidence log: every figure has a number, a file name, what it shows and where you will use it.
☐ Rename or move screenshots so the numbers match the log.
☐ Hashes recorded: sample (SHA-256) and M57-Jean image (SHA-256).
☐ Copy outputs: triage report, YARA rule, nmap, Nikto, dirb, Nessus export, SQLMap output.
☐ Write 5 bullet notes for each of A and B using the 5 parts: what I did, why, evidence, alternative, remaining risk.
☐ Remove anything that must not be uploaded: the sample, real credentials, tokens.
☐ Count your screenshot pages: **5 pages of A4 maximum** if screenshots are requested.
☐ Check the brief for what must be uploaded each day. Upload what it asks. **Submission is final.**
☐ Confirm the upload shows as submitted.
☐ Save your work in two places.

Set up tomorrow

☐ Autopsy installed and opens.
☐ M57-Jean image hashed and ready.
☐ Write on a sticky note: “Friday: E Autopsy, C assurance, D legal and IR, F justification, upload by 14:30.”

If something breaks

Tool will not install: describe what you would run, show the closest evidence, say plainly what you did not run.
Sample will not run in the sandbox: use static results plus public reports for the family, and cite them.
Prototype will not start: run **`test_sync.sh`** from the README. If it still fails, use the test table from your Ex1 submission as your baseline and say so.
Nessus will not activate: skip it, say so, and rely on nmap, Nikto and Burp.